



# ELECTRÓNICA DIGITAL III

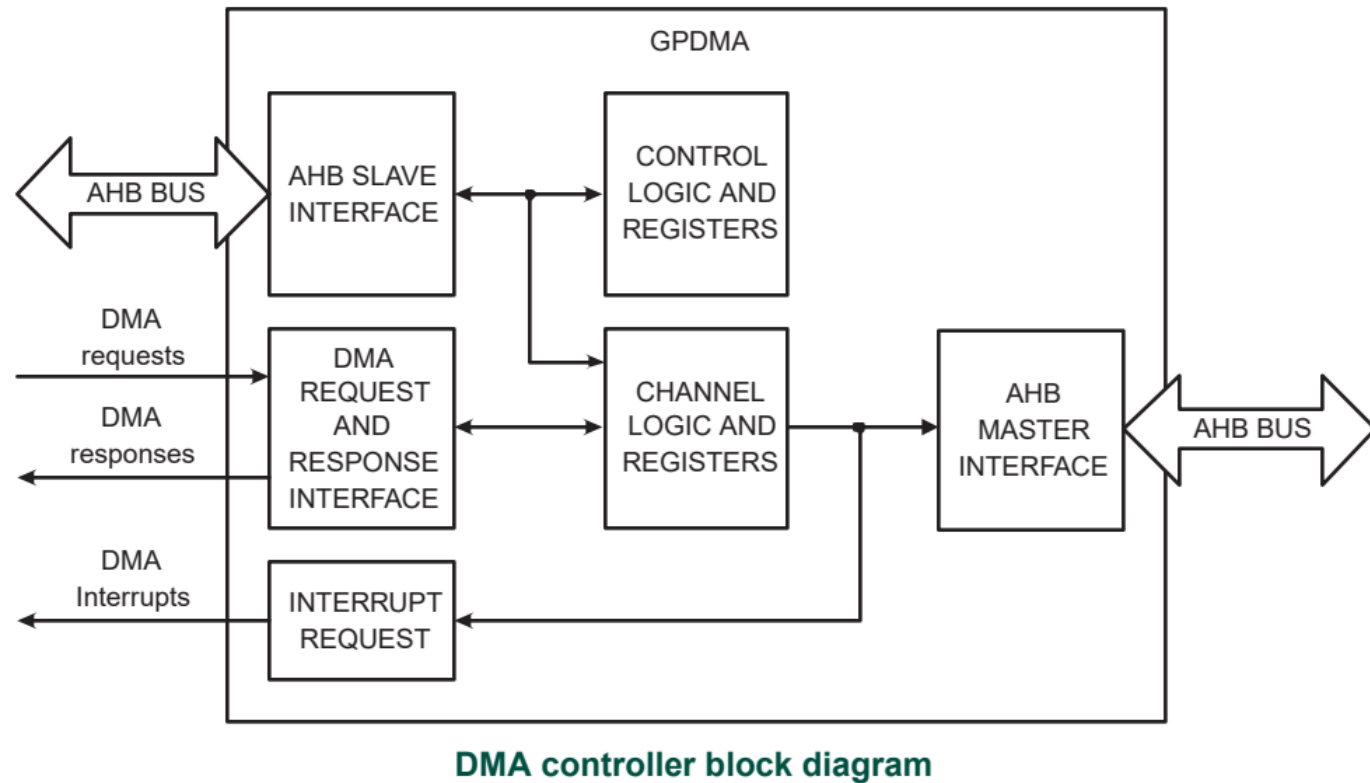
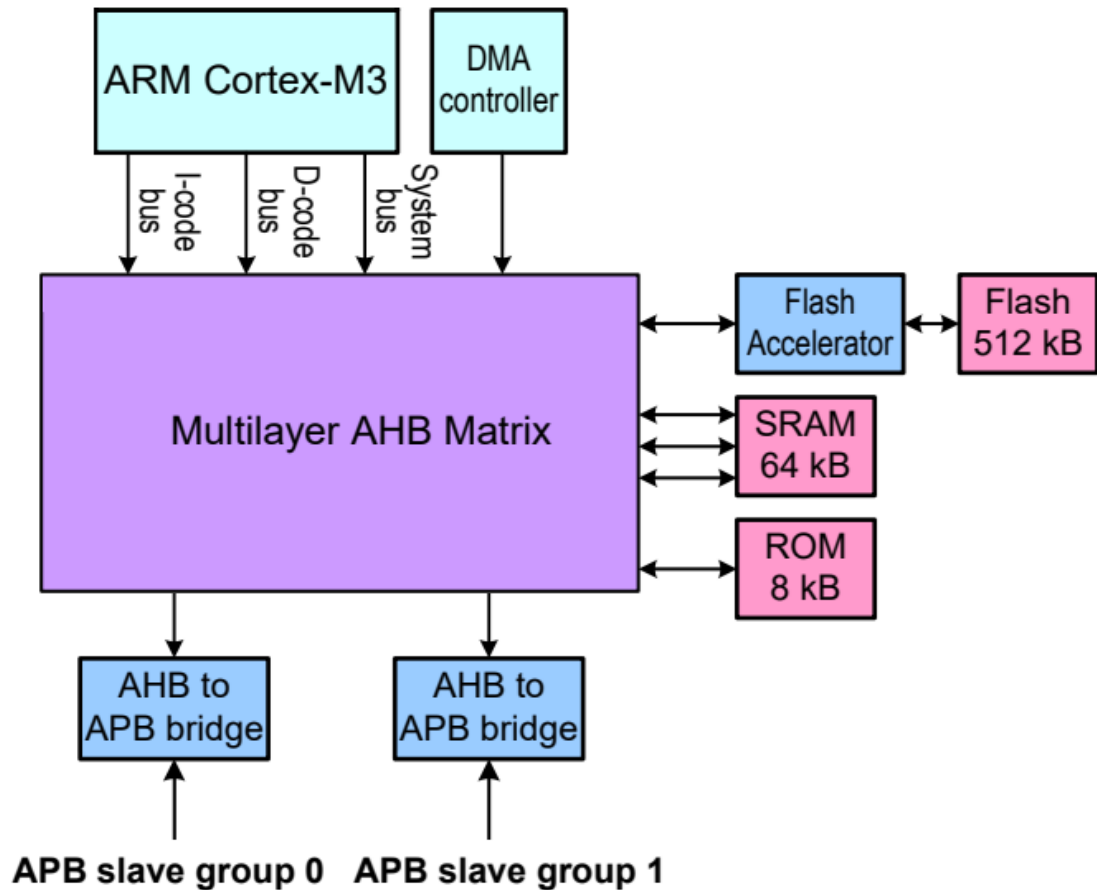
CLASE 8

Módulo GPDMA

Ing. Emiliano Migliore

# Módulo GPDMA:

El controlador de **GPDMA** permite gestionar de manera automática el acceso directo a la memoria, a través del bus **AHB**, para la transferencia de datos entre distintas regiones de memoria, desvinculando al core de dichas operaciones de transferencia.



**AHB (Advanced High-performance Bus)**  
**APB (Advanced Peripheral Bus)**

# Módulo GPDMA:

Este controlador cuenta con **8 canales**, permitiendo cada uno de ellos una comunicación unidireccional para las siguientes modalidades de transferencias de 8/16/32 bits:

$$P2M \text{ (Periférico a Memoria)} \Rightarrow \begin{cases} ADC \\ DAC \\ UART \\ I2S \\ SSP \end{cases} \rightarrow MEM$$

$$M2M \text{ (Memoria a Memoria)} \Rightarrow MEM \rightarrow MEM$$

$$M2P \text{ (Memoria a Periférico)} \Rightarrow MEM \rightarrow \begin{cases} ADC \\ DAC \\ UART \\ I2S \\ SSP \end{cases}$$

$$P2P \text{ (Periférico a Periférico)} \Rightarrow \begin{cases} ADC \\ DAC \\ UART \\ I2S \\ SSP \end{cases} \rightarrow \begin{cases} ADC \\ DAC \\ UART \\ I2S \\ SSP \end{cases}$$

### DMA request signal usage

| Transfer direction                          | Request generator                            | Flow controller |
|---------------------------------------------|----------------------------------------------|-----------------|
| Memory-to-peripheral                        | Peripheral                                   | DMA Controller  |
| Peripheral-to-memory                        | Peripheral                                   | DMA Controller  |
| Memory-to-memory                            | DMA Controller                               | DMA Controller  |
| Source peripheral to destination peripheral | Source peripheral and destination peripheral | DMA Controller  |

### Transfer type bits

| Bit value | Transfer type                               | Controller |
|-----------|---------------------------------------------|------------|
| 000       | Memory to memory                            | DMA        |
| 001       | Memory to peripheral                        | DMA        |
| 010       | Peripheral to memory                        | DMA        |
| 011       | Source peripheral to destination peripheral | DMA        |

# Módulo GPDMA:

Se dispone de **16 señales** de solicitudes de DMA que utilizan los periféricos para solicitar una transferencia de datos.

Estas señales indican si se requiere una transferencia de datos única o en ráfagas.

Por otra parte, las señales de respuesta de DMA indican si la transferencia iniciada por la señal de solicitud de DMA se ha completado.

Las prioridades de los canales son fijas, siendo el canal 0 el de máxima prioridad y el canal 7 el de menor prioridad. Si el controlador de DMA esta transfiriendo datos para el canal de menor prioridad y luego el canal de mayor prioridad se activa, se completa el número de transferencias en el canal de menor prioridad antes de cambiar la transferencia al canal de mayor prioridad.

Se puede generar una interrupción en el core ante una finalización exitosa de transferencia (INTTC) o ante un error de transferencia de datos (INTERR).

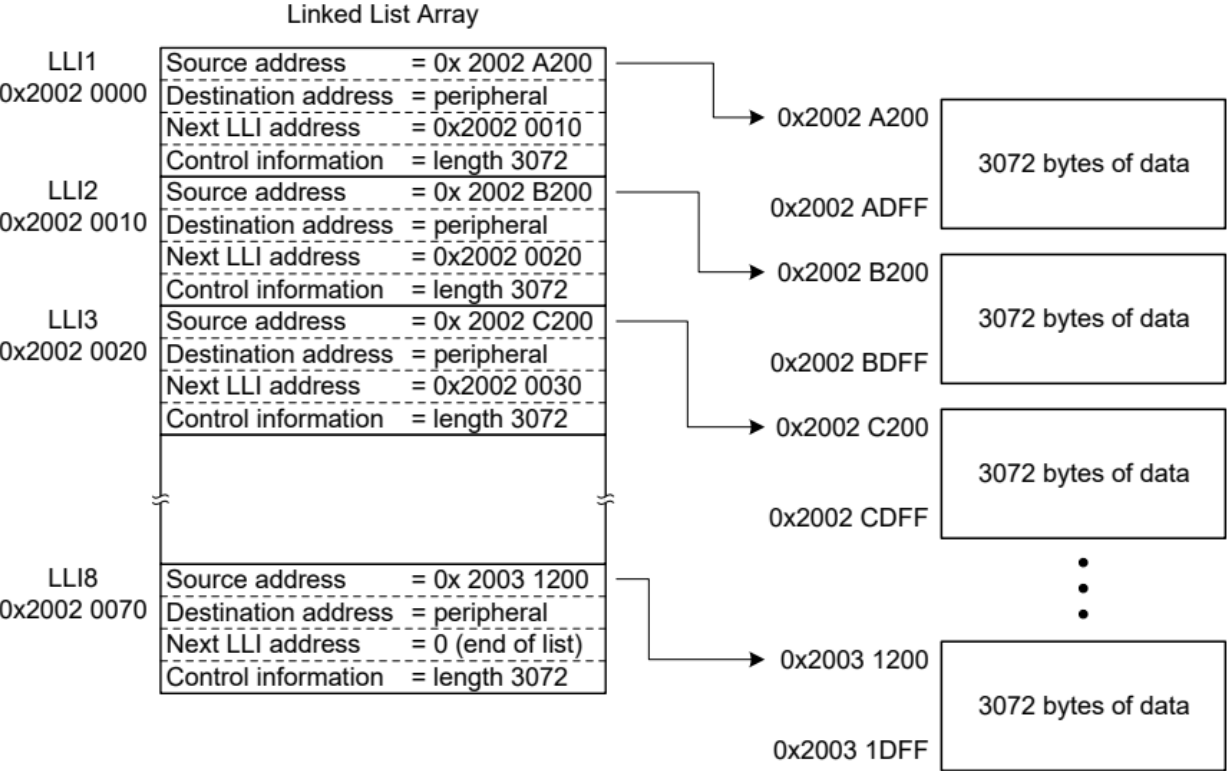
## DMA Connections

| Peripheral Function | DMA Single Request Input (DMACSREQ) | DMA Burst Request Input (DMACBREQ) | DMA Request Signal                        |
|---------------------|-------------------------------------|------------------------------------|-------------------------------------------|
| SSP0 Tx             | 0                                   | 0                                  | Dedicated DMA requests                    |
| SSP0 Rx             | 1                                   | 1                                  | Dedicated DMA requests                    |
| SSP1 Tx             | 2                                   | 2                                  | Dedicated DMA requests                    |
| SSP1 Rx             | 3                                   | 3                                  | Dedicated DMA requests                    |
| ADC                 | 4                                   | 4                                  | ADC interrupt request <a href="#">[1]</a> |
| I²S channel 0       | -                                   | 5                                  | Dedicated DMA request                     |
| I²S channel 1       | -                                   | 6                                  | Dedicated DMA request                     |
| DAC                 | -                                   | 7                                  | Dedicated DMA request                     |
| UART0 Tx / MAT0.0   | -                                   | 8                                  | Dedicated DMA requests                    |
| UART0 Rx / MAT0.1   | -                                   | 9                                  | Dedicated DMA requests                    |
| UART1 Tx / MAT1.0   | -                                   | 10                                 | Dedicated DMA requests                    |
| UART1 Rx / MAT1.1   | -                                   | 11                                 | Dedicated DMA requests                    |
| UART2 Tx / MAT2.0   | -                                   | 12                                 | Dedicated DMA requests                    |
| UART2 Rx / MAT2.1   | -                                   | 13                                 | Dedicated DMA requests                    |
| UART3 Tx / MAT3.0   | -                                   | 14                                 | Dedicated DMA requests                    |
| UART3 Rx / MAT3.1   | -                                   | 15                                 | Dedicated DMA requests                    |

[1] Generates an interrupt and/or DMA request depending on software setup.

# Módulo GPDMA:

La dispersión/recopilación se admite mediante el uso de **Listas Enlazadas** o **LLI (Linker List Items)**, son una serie de estructuras que se configuran de manera que una estructura LLI apunta a la siguiente estructura en la secuencia. Cada estructura LLI contiene información sobre una transferencia de DMA, incluyendo la dirección de origen, la dirección de destino, el tamaño de la transferencia, y el ancho de los datos. La ventaja de las listas vinculadas es que permiten encadenar múltiples transferencias de DMA en donde la fuente y las áreas de destino no tienen que ocupar áreas contiguas en la memoria. Una vez que se completa una transferencia, la unidad de control de DMA puede automáticamente avanzar al siguiente elemento de la lista y continuar con la siguiente transferencia sin necesidad de intervención adicional del core.



The first LLI, stored at 0x2002 0000, defines the first block of data to be transferred, which is the data stored from address 0x2002 A200 to 0x2002 ADFF:

- Source start address 0x2002 A200.
- Destination address set to the destination peripheral address.
- Transfer width, word (32-bit).
- Transfer size, 3072 bytes (0XC00).
- Source and destination burst sizes, 16 transfers.
- Next LLI address, 0x2002 0010.

The second LLI, stored at 0x2002 0010, describes the next block of data to be transferred:

- Source start address 0x2002 B200.
- Destination address set to the destination peripheral address.
- Transfer width, word (32-bit).
- Transfer size, 3072 bytes (0xC00).
- Source and destination burst sizes, 16 transfers.
- Next LLI address, 0x2002 0020.

# Ejemplo 1: Transferencia P2M para ADC con LLI Simple vía GPDMA

```
1  /*
2   * @brief  Configuración del Periférico ADC para Transferencia M2P con LLI Simple vía GPDMA
3   */
4
5  /* Directivas de Inclusión */
6  #include "LPC17xx.h"
7  #include "lpc17xx_gpdma.h"
8
9  /* Definición de Constantes y Macros */
10 #define ADC_BUFFER0_START  0x2007C000
11 #define ADC_BUFFER0_END    0x2007E000 //→0x7C000+((0x80000-0x7C000)/2)=0x7C000+0x2000=0x7E000
12 #define ADC_BUFFER0_SIZE  ((ADC_BUFFER0_END - ADC_BUFFER0_START)/sizeof(uint32_t)) //→0x2000/4[bytes] =0x800=2048
13
14 /* Definición de Variables Globales */
15 volatile uint32_t *adc_samples = (volatile uint32_t *)ADC_BUFFER0_START;
16 volatile bool ADC_FLAG = false;
17
18 /* Prototipos de Funciones */
19 void cfgDMA(void);
20 void storeADCsamples(void);
21
22 /* Función Principal */
23 int main(void)
24 {
25     //cfgADC();
26     cfgDMA();
27
28     while(1)
29     {
30         if(ADC_FLAG == true) storeADCsamples();
31     }
32     return 0;
33 }
34
```

# Ejemplo 1: Transferencia P2M para ADC con LLI Simple vía GPDMA

```
35  /* Definiciones de Funciones */
36  void storeADCsamples(void)
37  {
38      GPDMA_ChannelCmd(LPC_GPDMA_CH0, ENABLE);           //Iniciación de Envío de Datos por Canal 0 de DMA
39      //LPC_GPDMA_CH0->DMACCRConfig |= (1<<0);          //Habilitación del Canal 0 de DMA
40      return;
41  }
42
43  void cfgDMA(void)
44  {
45      GPDMA_LLI_Type cfgADC_LLI0_CH0;                    //Estructura para Lista Enlazada para canal 0 del DMA
46      GPDMA_Channel_CFG_Type cfgADC_DMA_CH0;              //Estructura de Configuración para canal 0 del DMA
47
48      NVIC_DisableIRQ(DMA_IRQn);                          //Deshabilitación de Interrupción por DMA
49      GPDMA_Init();                                         //Iniciación del Controlador GPDMA
50      //LPC_SC->PCON |= (1<<29);                           //Encendido del Periférico (GPDMA ON) - [Default: OFF]
51      //LPC_GPDMA_CH0->DMACCRConfig = 0;                   //Reset de Configuración para el Canal 0
52      //LPC_GPDMA_CH1->DMACCRConfig = 0;                   //Reset de Configuración para el Canal 1
53      //LPC_GPDMA_CH2->DMACCRConfig = 0;                   //Reset de Configuración para el Canal 2
54      //LPC_GPDMA_CH3->DMACCRConfig = 0;                   //Reset de Configuración para el Canal 3
55      //LPC_GPDMA_CH4->DMACCRConfig = 0;                   //Reset de Configuración para el Canal 4
56      //LPC_GPDMA_CH5->DMACCRConfig = 0;                   //Reset de Configuración para el Canal 5
57      //LPC_GPDMA_CH6->DMACCRConfig = 0;                   //Reset de Configuración para el Canal 6
58      //LPC_GPDMA_CH7->DMACCRConfig = 0;                   //Reset de Configuración para el Canal 7
59      //LPC_GPDMA->DMACIntTCClear = 0xFF;                  //Limpia Flag de Interrupción de Cuenta Terminal para todos los Canales
60      //LPC_GPDMA->DMACIntErrClr = 0xFF;                    //Limpia Flag de Interrupción de Error Terminal para todos los Canales
61
62
63      cfgADC_LLI0_CH0.SrcAddr = (uint32_t)&(LPC_ADC->ADDR0); //Dirección de Fuente de Datos (Periferico)
64      cfgADC_LLI0_CH0.DstAddr = (uint32_t)adc_samples;       //Dirección de Destino de Datos (Memoria)
65      cfgADC_LLI0_CH0.NextLLI = (uint32_t)&cfgADC_LLI0_CH0; //Próxima Dirección de Destino de Datos: LLI Repetitiva
66      cfgADC_LLI0_CH0.Control = (ADC_BUFFER_SIZE<<0)        //Tamaño de la Transferencia de la LLI
67      | (2<<18)                                                //Ancho de la Transferencia de Fuente (32-bit)
68      | (2<<21)                                                //Ancho de la Transferencia de Destino (32-bit)
69      | (1<<27)                                                //DI=0: Sin Incremento de la Dirección de Destino
70      & ~(1 << 26);                                           //SI=1: Con Incremento de la Dirección de Fuente
```



# Ejemplo 1: Transferencia P2M para ADC con LLI Simple vía GPDMA

```
71
72     cfgADC_DMA_CH0.ChannelNum = 0;
73     //LPC_GPDMA->DMACEnbldChns |= (1<<0);
74     cfgADC_DMA_CH0.SrcConn = GPDMA_CONN_ADC;
75     //LPC_GPDMA_CH0->DMACSrcAddr = (uint32_t)(LPC_ADC->ADDR0);
76     cfgADC_DMA_CH0.DstConn = 0;
77     cfgADC_DMA_CH0.SrcMemAddr = 0;
78     cfgADC_DMA_CH0.DstMemAddr = (uint32_t)adc_samples;
79     //LPC_GPDMA_CH0->DMACDestAddr = (uint32_t)adc_samples;
80     cfgADC_DMA_CH0.TransferType = GPDMA_TRANSFERTYPE_P2M;
81     cfgADC_DMA_CH0.TransferSize = ADC_BUFFER0_SIZE;
82     cfgADC_DMA_CH0.TransferWidth = 0;
83     //LPC_GPDMA_CH0->DMACControl |= ((1&0xFFF)<<0);
84     //LPC_GPDMA_CH0->DMACControl |= ((0x04&0x07)<<12);
85     //LPC_GPDMA_CH0->DMACControl |= ((0x04&0x07)<<15);
86     //LPC_GPDMA_CH0->DMACControl |= ((0x02&0x07)<<18);
87     //LPC_GPDMA_CH0->DMACControl |= ((0x02&0x07)<<21);
88     //LPC_GPDMA_CH0->DMACControl |= ((0x01<<27);
89     //LPC_GPDMA_CH0->DMACControl |= ((0x01<<31);
90     cfgADC_DMA_CH0.DMALLI = (uint32_t)&cfgADC_DMA_CH0;
91     //LPC_GPDMA_CH0->DMACCLLI = (uint32_t)&cfgADC_DMA_CH0;
92
93     GPDMA_Setup(&cfgADC_DMA_CH0);
94     //LPC_GPDMA->DMACIntTCClear |= (1<<0);
95     //LPC_GPDMA->DMACIntErrClr |= (1<<0);
96     //LPC_GPDMA_CH0->DMACCCConfig = 0;
97     //LPC_GPDMA_CH0->DMACCCControl = 0;
98     //LPC_SC->DMAREQSEL |= (1<<4)
99     //LPC_GPDMA->DMAConfig |= (1<<0)
100    //LPC_GPDMA_CH0->DMACCCConfig|= ((0x02&0x07)<<11);
101    //LPC_GPDMA_CH0->DMACCCConfig|= ((0x04&0x1F)<<1);
102
103    return;
104 }
105
```

```
//Selección del Canal de Transferencia
//EnabledChannels[0]=1: Habilitación del Canal 0 de DMA
//Periférico de Fuente de Datos (ADC)
//Asigna Dirección de Fuente de Datos
//Periférico de Destino de Datos (Sin Uso)
//Dirección de Fuente de Datos en Memoria (Sin Uso)
//Dirección de Destino de Datos en Memoria
//Asigna Dirección de Destino de Datos
//Tipo de Transferencia (Periférico a Memoria)
//Tamaño de Transferencia de Datos
//Ancho de Transferencia de Datos (Sin Uso)
//TransferSize=1: Tamano de Transferencia
//SBSIZE=001: Tamano de Rafaga de Fuente (4)
//DBSIZE=001: Tamano de Rafaga de Destino (4)
//SWidth=010: Ancho de Transferencia de Fuente (32-bit)
//DWidth=010: Ancho de Transferencia de Destino (32-bit)
//DI=1: Incremento de la Direccion de Destino
//I=1: Habilitación de Interrupción por Cuenta Terminal
//Lista Enlazada Inicial
//Asigna Dirección Inicial de la Lista Enlazada

//Configuración del Canal 0 de DMA
//IntCClear[0]=1: Limpia Flag de Interrupción de Recuento Terminal del Canal 0
//IntErrClr[0]=1: Limpia Flag de Interrupción de Error Terminal del Canal 0
//Reset de Configuración para el Canal 0
//Reset de Configuración de Control para el Canal 0
//Reconfigura la Solicitud de DMA para el Periférico Fuente
//Habilitación del Controlador DMA
//TransferType=010: Transferencia P2M
//SrcPeripheral=4: ADC como Periférico Fuente
```



## Ejemplo 2: Transferencia M2P para DAC con LLI Simple vía GPDMA e ISR

```
1  /*
2   * @brief   Configuración del Periférico DAC para Transferencia M2P con LLI Simple e ISR vía GPDMA
3   */
4
5   /* Directivas de Inclusión */
6   #include "LPC17xx.h"
7   #include "lpc17xx_gpdma.h"
8
9   /* Definición de Constantes y Macros */
10  #define DAC_BUFFER_START 0x20080000 //WaveForm
11  #define NUM_WAVE_SAMPLES 382
12
13  /* Definición de Variables Globales */
14  volatile uint32_t *dac_samples = (volatile uint32_t *)DAC_BUFFER_START;
15  volatile bool DAC_FLAG = false;
16  volatile uint32_t DMA_CH0_TC = 0; //Flag de Cuenta Terminal para Canal 0 de DMA
17  volatile uint32_t DMA_CH0_Err = 0; //Flag de Error Terminal para Canal 0 de DMA
18
19  /* Prototipos de Funciones */
20  void cfgDMA(void);
21  void loadDACwave(void);
22
23  /* Función Principal */
24  int main(void) {
25
26      //cfgDAC();
27      cfgDMA();
28
29      while(1)
30      {
31          if(DAC_FLAG == true) loadDACwave();
32      }
33      return 0;
34  }
```

## Ejemplo 2: Transferencia M2P para DAC con LLI Simple vía GPDMA e ISR

```
36  /* Definiciones de Funciones */
37  void loadDACwave(void)
38  {
39      GPDMA_ChannelCmd(LPC_GPDMA0, ENABLE);           //Iniciación de Envío de Datos por Canal 0 de DMA
40      NVIC_EnableIRQ(DMA_IRQn);                       //Habilitación de Interrupción por DMA
41      return;
42  }
43
44  void cfgDAC(void) {
45      uint32_t countDAC = 65535;                       //Número de Cuentas para el Timer Dedicado al DAC
46      DAC_CONVERTER_CFG_Type cfgDAC;
47
48      cfgDAC.CNT_ENA = SET;                             //Habilitación de TIMER Dedicado al DAC
49      cfgDAC.DMA_ENA = SET;                             //Habilitación de DAC por DMA
50
51      DAC_Init(LPC_DAC);                               //Iniciación del DAC
52      //LPC_SC->PCLKSEL0  &= ~(0x03<<22);             //PCLK_DAC=CCLK/4
53      //LPC_DAC->DACR &= ~(1<<16);                     //BIAS=0: Tiempo de Establecimiento de 1us (max)
54      DAC_SetBias(LPC_DAC,DAC_MAX_CURRENT_350uA);
55      //LPC_DAC->DACR |= (1<<16);                       //BIAS=1: Tiempo de Establecimiento de 2.5us (max)
56
57      DAC_ConfigDAConverterControl(LPC_DAC, &cfgDAC);  //Configuración del DAC
58      //LPC_DAC->DACCTRL |= (1<<2);                     //CNT_ENA=1: Habilitación de TIMER Dedicado al DAC
59      //LPC_DAC->DACCTRL |= (1<<3);                     //DMA_ENA=1: Habilitación de DAC por DMA
60
61      DAC_SetDMATimeOut(LPC_DAC,countDAC);             //Establecimiento del Valor de Recarga para el Timer Dedicado
62      //LPC_DAC->DACNTVAL = (countDAC&0xFFFF);         //DACNTVAL=65535[cuentas]
63      return;
64  }
65
```

$$\text{Sean } \begin{cases} t_{DAC_{max}} = PCLK^{-1} = 25MHz^{-1} = 40ns \\ n_{DAC_{max}} = (2^{16} - 1) = \boxed{65535} \end{cases}$$

$$\begin{cases} 1 \text{ cuenta} \rightarrow 40ns \\ 65535 \text{ cuentas} \rightarrow x = 2,62ms \end{cases}$$

$$\begin{cases} 2,62ms \rightarrow 1 \text{ muestra} \\ 1s \rightarrow \boxed{x \cong 382 \text{ muestras}} \end{cases}$$

## Ejemplo 2: Transferencia M2P para DAC con LLI Simple vía GPDMA e ISR

```
66 void cfgDMA(void)
67 {
68     GPDMA_LLI_Type cfgDAC_LLI0_CH0;
69     GPDMA_Channel_CFG_Type cfgDAC_DMA_CH0;
70
71     NVIC_DisableIRQ(DMA_IRQn);
72     GPDMA_Init();
73
74     cfgDAC_LLI0_CH0.SrcAddr = (uint32_t)dac_samples;
75     cfgDAC_LLI0_CH0.DstAddr = (uint32_t)&(LPC_DAC->DACR);
76     cfgDAC_LLI0_CH0.NextLLI = (uint32_t)&cfgDAC_LLI0_CH0;
77     cfgDAC_LLI0_CH0.Control = (NUM_WAVE_SAMPLES<<0)
78                               | (2<<18)
79                               | (2<<21)
80                               | (1<<26)
81                               & ~(1 << 27);
82
83     cfgDAC_DMA_CH0.ChannelNum = 0;
84     cfgDAC_DMA_CH0.SrcConn = 0;
85     cfgDAC_DMA_CH0.DstConn = GPDMA_CONN_DAC;
86     cfgDAC_DMA_CH0.SrcMemAddr = (uint32_t)dac_samples;
87     cfgDAC_DMA_CH0.DstMemAddr = 0;
88     cfgDAC_DMA_CH0.TransferType = GPDMA_TRANSFERTYPE_M2P;
89     cfgDAC_DMA_CH0.TransferSize = NUM_WAVE_SAMPLES;
90     cfgDAC_DMA_CH0.TransferWidth = 0;
91     cfgDAC_DMA_CH0.DMALLI = (uint32_t)&cfgDAC_LLI0_CH0;
92     GPDMA_Setup(&cfgDAC_DMA_CH0);
93
94     return;
95 }
96
```

//Estructura para Lista Enlazada para canal 0 del DMA  
//Estructura de Configuración para canal 0 del DMA  
  
//Deshabilitación de Interrupción por DMA  
//Inicialización del Controlador GPDMA  
  
//Dirección de Fuente de Datos (Memoria)  
//Dirección de Destino de Datos (Periferico)  
//Próxima Dirección de Destino de Datos: LLI Repetitiva  
//Tamaño de la Transferencia de la LLI  
//Ancho de la Transferencia de Fuente (32-bit)  
//Ancho de la Transferencia de Destino (32-bit)  
//SI=1: Con Incremento de la Dirección de Fuente  
//DI=0: Sin Incremento de la Dirección de Destino  
  
//Selección del Canal de Transferencia  
//Periférico de Fuente de Datos (Sin Uso)  
//Periférico de Destino de Datos (DAC)  
//Dirección de Fuente de Datos en Memoria  
//Dirección de Destino de Datos en Memoria (Sin Uso)  
//Tipo de Transferencia (Memoria a Periférico)  
//Tamaño de Transferencia de Datos  
//Ancho de Transferencia de Datos (Sin Uso)  
//Lista Enlazada Inicial  
//Configuración del Canal 0 de DMA

## Ejemplo 2: Transferencia M2P para DAC con LLI Simple vía GPDMA e ISR

```
97 void DMA_IRQHandler (void){
98     if(GPDMA_IntGetStatus(GPDMA_STAT_INT, 0)){           //Verifica Interrupción por Canal 0 de DMA
99         //if(LPC_GPDMA->DMACIntStat & ((1<<0)&0xFF)){
100         if(GPDMA_IntGetStatus(GPDMA_STAT_INTTC, 0)){     //Verifica Interrupción por Cuenta Terminal (1 List)
101             //if(LPC_GPDMA->DMACIntTCStat & ((1<<0)&0xFF)){
102
103                 //Ejecuta ISR: DMA_CH0_TC++;
104                 GPDMA_ClearIntPending (GPDMA_STATCLR_INTTC, 0); //Limpia Bandera de Interrupción por Cuenta Terminal
105                 //LPC_GPDMA->DMACIntTCClear |= ((1<<0)&0xFF);
106             //}
107         }
108         if(GPDMA_IntGetStatus(GPDMA_STAT_INTERR, 0)){     //Verifica Interrupción por Error Terminal
109             //if(LPC_GPDMA->DMACIntErrStat & ((1<<0)&0xFF)){
110
111                 //Ejecuta ISR: DMA_CH0_Err++;
112                 GPDMA_ClearIntPending (GPDMA_STATCLR_INTERR, 7); //Limpia Bandera de Interrupción por Error Terminal
113                 //LPC_GPDMA->DMACIntErrClear |= ((1<<0)&0xFF);
114             //}
115         }
116     //}
117 }
118 return;
119 }
120
```

# Ejemplo 3: Transferencia M2P para DAC con LLI Multiple vía GPDMA e ISR

```
1  /*
2   * @brief   Configuración del Periférico DAC para Transferencia M2P con LLI Multiple e ISR vía GPDMA
3   */
4
5  /* Directivas de Inclusión */
6  #include "LPC17xx.h"
7  #include "lpc17xx_gpdma.h"
8
9  /* Definición de Constantes y Macros */
10 #define DAC_BUFFER0_START 0x2007C000
11 #define DAC_BUFFER1_START 0x2007D000
12 #define DAC_BUFFER2_START 0x2007E000
13 #define NUM_WAVE_SAMPLES 1024
14
15 /* Definición de Variables Globales */
16 volatile uint32_t *dac_samples0 = (volatile uint32_t *)DAC_BUFFER0_START; //WaveForm0
17 volatile uint32_t *dac_samples1 = (volatile uint32_t *)DAC_BUFFER1_START; //WaveForm1
18 volatile uint32_t *dac_samples2 = (volatile uint32_t *)DAC_BUFFER2_START; //WaveForm2
19 volatile bool DAC_FLAG = false;
20 volatile uint32_t DMA_CH0_TC = 0; //Flag de Cuenta Terminal para Canal 0 de DMA
21 volatile uint32_t DMA_CH0_Err = 0; //Flag de Error Terminal para Canal 0 de DMA
22
23 /* Prototipos de Funciones */
24 void cfgDMA(void);
25 void loadDACwaves(void);
26
27 /* Función Principal */
28 int main(void){
29
30     //cfgDAC();
31     cfgDMA();
32
33     while(1)
34     {
35         if(DAC_FLAG == true) loadDACwaves();
36     }
37     return 0;
38 }
39
```

# Ejemplo 3: Transferencia M2P para DAC con LLI Multiple vía GPDMA e ISR

```
40  /* Definiciones de Funciones */
41  void loadDACwaves(void)
42  {
43      GPDMA_ChannelCmd(LPC_GPDMA0, ENABLE);           //Inicialización de Envío de Datos por Canal 0 de DMA
44      NVIC_EnableIRQ(DMA_IRQn);                       //Habilitación de Interrupción por DMA
45      return;
46  }
47
48  void cfgDMA(void)
49  {
50      GPDMA_LLI_Type cfgDAC_LLI0_CH0;                 //Estructura para Lista Enlazada 0 para canal 0 del DMA
51      GPDMA_LLI_Type cfgDAC_LLI1_CH0;                 //Estructura para Lista Enlazada 1 para canal 0 del DMA
52      GPDMA_LLI_Type cfgDAC_LLI2_CH0;                 //Estructura para Lista Enlazada 2 para canal 0 del DMA
53      GPDMA_Channel_CFG_Type cfgDAC_DMA_CH0;          //Estructura de Configuración para canal 0 del DMA
54
55      NVIC_DisableIRQ(DMA_IRQn);                       //Deshabilitación de Interrupción por DMA
56      GPDMA_Init();                                    //Inicialización del Controlador GPDMA
57
58      cfgDAC_LLI0_CH0.SrcAddr = (uint32_t)dac_samples0; //Dirección de Fuente de Datos (Memoria)
59      cfgDAC_LLI0_CH0.DstAddr = (uint32_t)&(LPC_DAC->DACR); //Dirección de Destino de Datos (Periferico)
60      cfgDAC_LLI0_CH0.NextLLI = (uint32_t)&cfgDAC_LLI1_CH0; //Próxima Dirección de Destino de Datos: LLI Repetitiva
61      cfgDAC_LLI0_CH0.Control = (NUM_WAVE_SAMPLES<<0) //Tamaño de la Transferencia de la LLI0
62      | (2<<18) //Ancho de la Transferencia de Fuente (32-bit)
63      | (2<<21) //Ancho de la Transferencia de Destino (32-bit)
64      | (1<<26) //SI=1: Con Incremento de la Dirección de Fuente
65      & ~(1 << 27); //DI=0: Sin Incremento de la Dirección de Destino
66      cfgDAC_LLI1_CH0.SrcAddr = (uint32_t)dac_samples1; //Dirección de Fuente de Datos (Memoria)
67      cfgDAC_LLI1_CH0.DstAddr = (uint32_t)&(LPC_DAC->DACR); //Dirección de Destino de Datos (Periferico)
68      cfgDAC_LLI1_CH0.NextLLI = (uint32_t)&cfgDAC_LLI2_CH0; //Próxima Dirección de Destino de Datos: LLI Repetitiva
69      cfgDAC_LLI1_CH0.Control = (NUM_WAVE_SAMPLES<<0) //Tamaño de la Transferencia de la LLI1
70      | (2<<18) //Ancho de la Transferencia de Fuente (32-bit)
71      | (2<<21) //Ancho de la Transferencia de Destino (32-bit)
72      | (1<<26) //SI=1: Con Incremento de la Dirección de Fuente
73      & ~(1 << 27); //DI=0: Sin Incremento de la Dirección de Destino
74      cfgDAC_LLI2_CH0.SrcAddr = (uint32_t)dac_samples2; //Dirección de Fuente de Datos (Memoria)
75      cfgDAC_LLI2_CH0.DstAddr = (uint32_t)&(LPC_DAC->DACR); //Dirección de Destino de Datos (Periferico)
76      cfgDAC_LLI2_CH0.NextLLI = (uint32_t)&cfgDAC_LLI0_CH0; //Próxima Dirección de Destino de Datos: LLI Repetitiva
77      cfgDAC_LLI2_CH0.Control = (NUM_WAVE_SAMPLES<<0) //Tamaño de la Transferencia de la LLI2
78      | (2<<18) //Ancho de la Transferencia de Fuente (32-bit)
79      | (2<<21) //Ancho de la Transferencia de Destino (32-bit)
80      | (1<<26) //SI=1: Con Incremento de la Dirección de Fuente
81      & ~(1 << 27); //DI=0: Sin Incremento de la Dirección de Destino
82  }
```



# Ejemplo 3: Transferencia M2P para DAC con LLI Multiple vía GPDMA e ISR

```
83     cfgDAC_DMA_CH0.ChannelNum = 0;
84     cfgDAC_DMA_CH0.SrcConn = 0;
85     cfgDAC_DMA_CH0.DstConn = GPDMA_CONN_DAC;
86     cfgDAC_DMA_CH0.SrcMemAddr = cfgDAC_LLI0_CH0.SrcAddr;
87     cfgDAC_DMA_CH0.DstMemAddr = 0;
88     cfgDAC_DMA_CH0.TransferType = GPDMA_TRANSFERTYPE_M2P;
89     cfgDAC_DMA_CH0.TransferSize = NUM_WAVE_SAMPLES;
90     cfgDAC_DMA_CH0.TransferWidth = 0;
91     cfgDAC_DMA_CH0.DMALLI = (uint32_t)&cfgDAC_LLI0_CH0;
92     GPDMA_Setup(&cfgDAC_DMA_CH0);
93
94     return;
95 }
96
97 void DMA_IRQHandler (void){
98     if(GPDMA_IntGetStatus(GPDMA_STAT_INT, 0)){
99         if(GPDMA_IntGetStatus(GPDMA_STAT_INTTC, 0)){
100             //Ejecuta ISR: DMA_CH0_TC++;
101             GPDMA_ClearIntPending (GPDMA_STATCLR_INTTC, 0);
102         }
103         if(GPDMA_IntGetStatus(GPDMA_STAT_INTERR, 0)){
104             //Ejecuta ISR: DMA_CH0_Err++;
105             GPDMA_ClearIntPending (GPDMA_STATCLR_INTERR, 7);
106         }
107     }
108     return;
109 }
110 }
```

//Selección del Canal de Transferencia  
//Periférico de Fuente de Datos (Sin Uso)  
//Periférico de Destino de Datos (DAC)  
//Dirección de Fuente de Datos en Memoria  
//Dirección de Destino de Datos en Memoria (Sin Uso)  
//Tipo de Transferencia (Memoria a Periférico)  
//Tamaño de Transferencia de Datos  
//Ancho de Transferencia de Datos (Sin Uso)  
//Lista Enlazada Inicial  
//Configuración del Canal 0 de DMA

//Verifica Interrupción por Canal 0 de DMA  
//Verifica Interrupción por Cuenta Terminal (1 List)  
//Limpia Bandera de Interrupción por Cuenta Terminal  
//Verifica Interrupción por Error Terminal  
//Limpia Bandera de Interrupción por Error Terminal

# Ejemplo 4: Transferencia M2M vía GPDMA

```
1  /*
2   * @brief   Configuración del Módulo GPDMA para Transferencia M2M sin LLI
3   */
4
5  /* Directivas de Inclusión */
6  #include "LPC17xx.h"
7  #include "lpc17xx_gpdma.h"
8
9  /* Definición de Constantes y Macros */
10 #define MEM_BUFFER_SRC_START 0x2007C000
11 #define MEM_BUFFER_DST_START 0x2007E000
12 #define MEM_BUFFER_SIZE ((MEM_BUFFER_DST_START - MEM_BUFFER_SRC_START)/sizeof(uint32_t)) ↪ 0x2000/4[bytes] =0x800=2048
13
14 /* Definición de Variables Globales */
15 uint32_t *Fuente = (uint32_t *)MEM_BUFFER_SRC_START;
16 uint32_t *Destino = (uint32_t *)MEM_BUFFER_DST_START;
17 volatile bool TRANSF_FLAG = false;
18
19 /* Prototipos de Funciones */
20 void cfgDMA(void);
21 void transferStoredData(void);
22
23 /* Función Principal */
24 int main(void){
25
26     cfgDMA();
27
28     while(1)
29     {
30         if(TRANSF_FLAG == true) transferStoredData();
31     }
32     return 0;
33 }
34
35
```

# Ejemplo 4: Transferencia M2M vía GPDMA

```
36  /* Definiciones de Funciones */
37  void transferStoredData(void)
38  {
39      GPDMA_ChannelCmd(LPC_GPDMA_CH7, ENABLE);           //Iniciación de Envío de Datos por Canal 7 de DMA
40      //LPC_GPDMA_CH7->DMACCRConfig |= (1<<7);          //Habilitación del Canal 7 de DMA
41      NVIC_EnableIRQ(DMA_IRQn);                          //Habilitación de Interrupción por DMA
42      return;
43  }
44
45  void cfgDMA(void)
46  {
47      GPDMA_Channel_CFG_Type cfgMEM_DMA_CH7;             //Estructura de Configuración para canal 7 del DMA
48
49      NVIC_DisableIRQ(DMA_IRQn);                          //Deshabilitación de Interrupción por DMA
50      GPDMA_Init();                                       //Iniciación del Controlador GPDMA
51
52      cfgMEM_DMA_CH7.ChannelNum = 7;                     //Selección del Canal de Transferencia
53      cfgMEM_DMA_CH7.SrcConn = 0;                        //Periférico de Fuente de Datos (Sin Uso)
54      cfgMEM_DMA_CH7.DstConn = 0;                        //Periférico de Destino de Datos (Sin Uso)
55      cfgMEM_DMA_CH7.SrcMemAddr = (uint32_t)Fuente;      //Dirección de Fuente de Datos en Memoria
56      //LPC_GPDMA_CH7->DMACSrcAddr = (uint32_t)Fuente;   //Asigna Dirección de Fuente de Datos
57      cfgMEM_DMA_CH7.DstMemAddr = (uint32_t)Destino;     //Dirección de Destino de Datos en Memoria
58      //LPC_GPDMA_CH7->DMACDestAddr = (uint32_t)Destino; //Asigna Dirección de Destino de Datos
59      cfgMEM_DMA_CH7.TransferType = GPDMA_TRANSFERTYPE_M2M; //Tipo de Transferencia (Memoria a Memoria)
60      cfgMEM_DMA_CH7.TransferSize = MEM_BUFFER_SIZE;    //Tamaño de Transferencia de Datos
61      cfgMEM_DMA_CH7.TransferWidth = GPDMA_WIDTH_WORD;  //Ancho de Transferencia de Datos
62      //LPC_GPDMA_CH7->DMACControl |= ((MEM_BUFFER_SIZE&0xFFF)<<0); //TransferSize=2048: Tamano de Transferencia
63      //LPC_GPDMA_CH7->DMACControl |= ((0x04&0x07)<<12); //SBSize=100: Tamano de Rafaga de Fuente (32)
64      //LPC_GPDMA_CH7->DMACControl |= ((0x04&0x07)<<15); //DBSize=100: Tamano de Rafaga de Destino (32)
65      //LPC_GPDMA_CH7->DMACControl |= ((0x02&0x07)<<18); //SWidth=010: Ancho de Transferencia de Fuente (32-bit)
66      //LPC_GPDMA_CH7->DMACControl |= ((0x02&0x07)<<21); //DWidth=010: Ancho de Transferencia de Destino (32-bit)
67      //LPC_GPDMA_CH7->DMACControl |= ((0x01<<26);      //SI=1: Incremento de la Direccion de Fuente
68      //LPC_GPDMA_CH7->DMACControl |= ((0x01<<27);      //DI=1: Incremento de la Direccion de Destino
69      //LPC_GPDMA_CH7->DMACControl |= ((0x01<<31);      //I=1: Habilitación de Interrupción por Cuenta Terminal
70      cfgMEM_DMA_CH7.DMALLI = 0;                         //Lista Enlazada Inicial (Sin Uso)
71      GPDMA_Setup(&cfgMEM_DMA_CH7);
72
73      return;
74  }
```

# Ejemplo 4: Transferencia M2M vía GPDMA

```
76 void DMA_IRQHandler (void){
77     if(GPDMA_IntGetStatus(GPDMA_STAT_INT, 7)){           //Verifica Interrupción por Canal 7 de DMA
78         if(GPDMA_IntGetStatus(GPDMA_STAT_INTTC, 7)){     //Verifica Interrupción por Cuenta Terminal (1 List)
79
80             //Ejecuta ISR: DMA_CH7_TC++;
81
82             /*** Tx Datos vía UART ***/
83
84             GPDMA_ClearIntPending (GPDMA_STATCLR_INTTC, 7); //Limpia Bandera de Interrupción por Cuenta Terminal
85         }
86         if(GPDMA_IntGetStatus(GPDMA_STAT_INTERR, 7)){     //Verifica Interrupción por Error Terminal
87
88             //Ejecuta ISR: DMA_CH7_Err++;
89             GPDMA_ClearIntPending (GPDMA_STATCLR_INTERR, 7); //Limpia Bandera de Interrupción por Error Terminal
90         }
91     }
92     return;
93 }
94
```

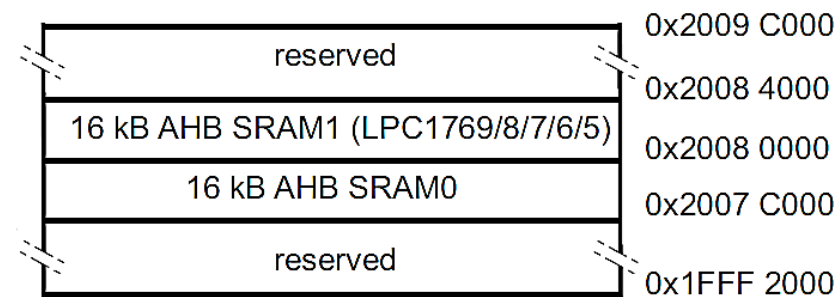
# Ejercicio Integrador (1/2)

Programar el microcontrolador LPC1769 en un código en lenguaje C, con una frecuencia de core cclk de 80MHz, para que mediante una interrupción externa en el pin P2.10 por flanco ascendente, se comande secuencialmente las siguientes funcionalidades:

Mediante el ADC se debe digitalizar una señal analógica cuyo ancho de banda es de 80[KHz], y su amplitud de pico máxima positiva es de 3,3V.

- Los datos deben ser guardados, cada 1 segundo, utilizando punteros (ADC\_POINTER).
- Los datos deben ser guardados, cada 1 segundo, utilizando el hardware GPDMA (ADC\_DMA).

En ambos casos, desde la primera mitad de la memoria SRAM ubicada en el bloque AHB SRAM-bank 0 de manera tal que permita almacenar todos los datos posibles que esta memoria permita, mediante un buffer circular conservando siempre las ultimas muestras.



Mediante el hardware GPDMA se debe transferir los datos desde la primera mitad de la memoria SRAM ubicada en el bloque AHB SRAM-bank 0 hacia la segunda mitad de dicha memoria, en donde se deben promediar todos los datos, y dicho valor digitalizado promediado debe ser transmitido por la UART0 en dos tramas de 1 byte que representan la parte entera y decimal del mismo.

## Ejercicio Integrador (2/2)

Mediante el DAC:

Se comienza a sacar las muestras del ADC por el DAC mediante el hardware GPDMA desde las posiciones originales de memoria (DAC\_DATA).

Se debe generar una función que represente la forma de onda de la figura, mediante 382 muestras, para ser reproducida por el DAC desde la memoria SRAM ubicada en el bloque AHB SRAM-bank 1 utilizando el hardware GPDMA de tal forma que se logre un periodo de 1s logrando la máxima resolución y máximo rango de tensión permitido. (Se debe indicar cual es el mínimo incremento de tensión de salida de dicha forma de onda). (DAC\_WAVE).

